

UNIT STUDY NOTES

Modules, Packages and Environments

How code becomes an importable project and how dependencies stay reproducible across machines. Covers the import system, package layout, and virtual environment isolation.

 5 source notebooks 48 topic sections extracted 7 study sections

Unit Overview

Welcome to the first lesson of Modules, Packages, and Environments — the topic where Python stops being just "your code" and becomes "your code plus the world's code". Everything you have written so far lived in a single file, starting from line one and running to the end. Real programs are never like that. A real project is a collection of files: the logic you wrote, the helpers your teammates wrote, and thousands of battle-tested functions written by programmers you have never met. The import system is the bridge that connects all of them.

We can think of imports like ordering lunch from a busy mess counter. The Python standard library is the kitchen: a big, well-organised space that ships with every Python install and needs no extra setup. `math` is the counter with the number machines, `random` is the counter with the dice and the lottery bowls, and `datetime` is the counter with the calendars and the clocks. Your job as the programmer is simply to reach across and take what you need — but HOW you take it changes your life in that kitchen.

In this notebook you will master:

1. What a module is, and how the standard library gives you a free toolbox.
2. The four classic import styles: `import X`, `import X as y`, `from X import name`, and the dangerous `from X import *`.
3. The `math` module in action: squares, factorials, pi, hypotenuse, greatest common divisor, and ceiling and floor.
4. The `random` module's seed: the trick that turns "random" into "repeatable".
5. The `datetime` module: subtracting dates, adding weeks, and reading a weekday name.

Along the way we will look inside a module with `dir()`, and we will nearly step on a landmine called the shadow trap — the mistake of naming your own file `math.py` and silently replacing Python's real one.

A module, at its core, is just a `.py` file full of reusable functions, classes, and variables. A library is a collection of such modules, and the standard library is the install-free collection that ships inside Python itself. By the end of this lesson you will be importing like a professional: confidently, deliberately, and without polluting your own namespace. The mess-counter theme continues — let's grab lunch!

Welcome to the second topic of this unit: creating modules. So far you have been a *consumer* of Python's standard library — importing `math`, `random`, and `datetime` built by other people. Now the tables turn: you become a **producer**. You will write your own `.py` file, import it, inspect it, and control how much of it runs. This is the step where “Python programming” becomes “building software that other code can reuse.”

1. What you will learn in this notebook

1. **A module is just a file** — a `.py` file full of functions, classes, and variables. Writing one is no different from writing any script you have done so far; what changes is how Python finds and loads it.
2. **`sys.path` — Python's map to every module.** When you `import grade_utils`, Python searches `sys.path`, an ordered list of folders. If your file's folder is on that list, the import succeeds. We will insert a temp folder into `sys.path` live.
3. **`dir()` and docstrings — your module's self-description.** Every module can announce its own contents and explain its own functions.
4. **The `__name__ == "__main__"` guard — the two lives of a file.** The same file can behave as a quiet import library or as a loud standalone script, depending on how it is launched.
5. **`sys.modules` cache and `__pycache__` — behind-the-scenes machinery that explains why re-imports are cheap and why old code sometimes “sticks.”**

2. The big idea

Real projects reuse helper functions everywhere: grading marks, formatting dates, fetching data. Copy-pasting the same code into ten notebooks is a recipe for bugs — fix the logic in one place and nine copies stay wrong. The professional move is to write the helper once inside a module and `import` it wherever it is needed. That one decision turns scattered scripts into a maintainable library.

3. How the demos work

Because a notebook cannot easily create files next to itself, the demos write modules into temporary folders via `tempfile` + `sys.path.insert(0, tmp)`. This keeps your real workspace clean while letting you experience the full import cycle: Python finds the file, executes it bottom-to-top, and returns a module object whose names you can use immediately. Run each cell, watch the output, and notice how ordinary a “library” really is.

A single module solved the “reuse your functions” problem. But real projects grow large: hundreds of modules, thousands of functions. Dumped in one flat folder, that becomes chaos. Python's answer is a **package** — a folder of related modules with an `__init__.py` in it, forming a tidy family tree that mirrors how real projects are organised.

1. What you will learn in this notebook

1. **Package = folder + `__init__.py`** — the marker that turns an ordinary directory into an importable family of modules.
2. **Three import styles for packages** — `import pkg`, `from pkg import mod`, and dotted imports like `pkg.mod.func`.

3. **The power of `__init__.py`** — a package banner, plus *re-export shortcuts* so users can reach deep functions with a short name.
4. **Relative imports** — how submodules talk to each other inside the package with leading-dot paths like `from .billing import tax`.
5. **The dunder zoo** — `__name__`, `__doc__`, `__package__`, `__path__`, and the `__pycache__` folder, i.e. how a package identifies and presents itself.

2. The big idea — order in the house-cupboard

Picture your kitchen storage. Everything mixed in one drawer is a hotchpotch; labelled boxes — kitchen, study, tools — make every item predictable. A Python project is the same: instead of fifty loose files, we group them into packages. `dukan/` becomes a shop with a `billing` shelf and a `catalog` shelf; `study/` becomes a room with a `notes` notebook. The `__init__.py` in each folder is the index-card that says what that drawer contains and which shortcuts it offers.

3. How the demo works

All demos build the package entirely in a temporary folder using `tempfile` and plain file writes, then add the parent folder to `sys.path`. Since `__init__.py` executes on the very first import, you will hear the package announce itself with a banner — proof that loading a package is not free silence but a deliberate, scriptable event. Files are written exactly as they would appear in a real project, so everything you learn here transfers directly to a repository on disk.

Up to this point, everything you installed went into one global, shared space. For a few small experiments that works fine — but the moment you hold two real projects, the global space turns into a war zone. Project A needs `cheese_lib` v1, Project B needs the same-named `cheese_lib` v2, and a single shared folder can only hold one of them. Whichever installs last wins; the other project silently breaks. This is the classic dependency conflict, and the professional solution is the **virtual environment**: a private, isolated folder holding its own Python, its own `pip`, and its own libraries, completely separate from the system's global installation.

1. What you will learn here

1. **Why environments matter** — the dependency-isolation story, told through the kitchen war of two projects.
2. **The 3-step terminal mantra** — create → activate → work inside; leave with `deactivate`.
3. **requirements.txt — the project passport** — export installed packages with `pip freeze`, revive them anywhere with `pip install -r requirements.txt`.
4. **The venv anatomy** — a live `python -m venv` demo that shows you `bin/`, `lib/`, `pyvenv.cfg`, and the activation scripts.
5. **The isolation proof** — `sys.prefix != sys.base_prefix` printed as `True`, the mechanical formula that guarantees your env is genuinely separate.

2. How the demos work

The notebook runs real `venv` commands from inside Python using `subprocess`, building a throwaway environment in a temp folder. Because interacting with a shell `activate` script mid-notebook is awkward, the demos call the environment's own `python` binary directly — which is exactly what activation does behind the scenes. You will also get copy-paste cheat-sheets for both Linux/macOS and Windows terminals, because in everyday life you will `venv` from a shell, not a notebook.

3. The big idea

Every project gets its own kitchen: separate counters, separate libraries, separate pip life. Two projects needing conflicting versions coexist peacefully because each sees only its own kitchen. This one idea removes an entire class of “works on my machine” disasters and is standard practice in every Python workplace.

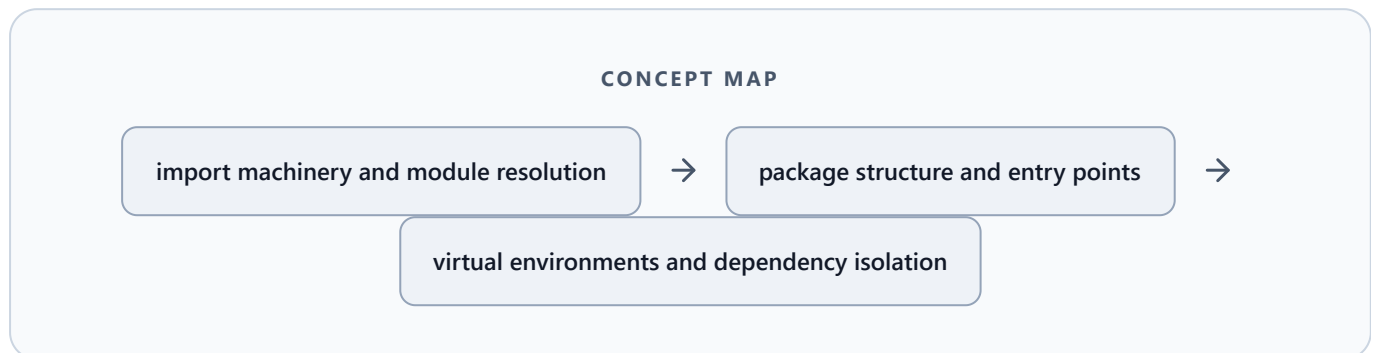
Prerequisites

→ Advanced Python

What you will be able to do

- ✓ Trace how the import system resolves a module to a file
- ✓ Lay out a package that others can install and import
- ✓ Create and use isolated virtual environments
- ✓ Keep dependencies declared and reproducible

What This Unit Covers



The sections below follow this arc, from foundational semantics to the engineering decisions that follow from them.

Core Concepts

The Python Import System — Making the Library Work for You

Real-World Hook: Hostel Mess Lunch 🍽️

How lunch is composed:

- Lift the **WHOLE** mess counter (everything!) → `import mess` (bread/lentils/vegetables, all together!)
- Only the target dish from one counter → `from mess import curry`
- Take everything without picking a dish → `from mess import *` (⚠️ risky!)

📖 Definition (interview-grade!)

Module = a `.py` file containing functions/classes/variables — import and reuse it! **Library/Package** = a family of modules. `math` / `random` / `datetime` = Python STANDARD-LIBRARY (an install-free zoo! 🦒)

1. The Four Import Styles

1.1 Style 1 — `import module` : The Whole Counter, Prefix Calls

The first and safest style is the plain `import module`. Think of it as lifting the *entire counter* over to your seat: every tool that counter offers is now available, but every tool is labelled with its counter name. You call `math.sqrt(625)` rather than a bare `sqrt(625)`.

Why does this matter? Because labels prevent collisions. Python files reuse names freely — you might have a variable called `sum` or a function called `random`, anything. If every imported function came without a label, your variables and the library's names would fight for the same name, and whichever was defined later would silently win.

The code below imports `math` and immediately puts it to work:

- `math.sqrt(625)` — the square root; 625 is a perfect square, so `25.0`.
- `math.factorial(6)` — `6! = 6 * 5 * 4 * 3 * 2 * 1 = 720`.
- `round(math.pi, 4)` — pi shortened to four decimals: `3.1416`.
- `math.hypot(3, 4)` — the hypotenuse of a right triangle with legs 3 and 4; Pythagoras gives `5.0`.
- `math.gcd(48, 180)` — the greatest common divisor, `12`.
- `math.ceil(9.2)` and `math.floor(9.2)` — rounding up to `10` and down to `9`; they always move to the nearest whole numbers above and below.

Notice the pattern: `import math` creates a **namespace**, and you reach inside it with the dot operator. Later on, this habit — always going through a label — is the single best defensive habit in Python.

PYTHON

```
# 🍌 STYLE-1: `import module` – pura-counter lift, naam-se-call!
import math

print("sqrt(625) :", math.sqrt(625))
print("factorial 6:", math.factorial(6))
print("pi round-4 :", round(math.pi, 4))
print("hypot(3,4) :", math.hypot(3, 4))
print("gcd(48,180):", math.gcd(48, 180))
print("ceil 9.2 :", math.ceil(9.2), "| floor 9.2:", math.floor(9.2))
# Expected OUTPUT:
# sqrt(625) : 25.0
# factorial 6: 720
# pi round-4 : 3.1416
# hypot(3,4) : 5.0
# gcd(48,180): 12
# ceil 9.2 : 10 | floor 9.2: 9
```

1.2 Style 2 — `import module as alias` : Give It a Short Stamp

Typing `math.sqrt` all day is fine, but some module names are long. Python lets you attach a short nickname with `import module as alias`. After `import math as m`, the name `m` refers to the exact same module object — there is only ever one `math` loaded — and you call `m.hypot(5, 12)` exactly as you would call `math.hypot(5, 12)`.

The alias is your personal abbreviation, chosen by you, for you. It changes nothing about how Python resolves the import; it only changes the handle you use from that point on. Different files can even give the same module different aliases without conflict.

The same cell shows why `datetime` is a wonderful everyday module. Dates are objects, and objects know how to behave:

- `dt.date(2026, 9, 17)` builds a fixed date — no time, no timezone, just the calendar day.
- `.strftime("%A")` formats that date as the full weekday name, here "Thursday".
- Subtracting two dates gives a `timedelta`, and reading `.days` tells you how many calendar days lie between them — here `106` days from 17 September 2026 to 1 January 2027.
- Adding `dt.timedelta(weeks=2)` moves the date forward by fourteen days, landing on 1 October 2026.

This is the heart of library reuse: someone built the hard, fiddly calendar logic, and you get to say "add two weeks" in one readable line.

PYTHON

```
# 🍷 STYLE-2: `import module as alias` — short-stamp nickname!
import math as m
import datetime as dt

print("m.hypot      :", m.hypot(5, 12))
today = dt.date(2026, 9, 17)           # fixed-date (deterministic!) 📅
print("today       :", today, "| weekday:", today.strftime("%A"))
newyear = dt.date(2027, 1, 1)
print("days-left  :", (newyear - today).days, "(NY-2027 tak!)")
print("+2 weeks    :", today + dt.timedelta(weeks=2))
# Expected OUTPUT:
# m.hypot        : 13.0
# today          : 2026-09-17 | weekday: Thursday
# days-left      : 106 (NY-2027 tak!)
# +2 weeks       : 2026-10-01
```

1.3 Style 3 — `from module import name` : Grab Only the Dish You Need

Sometimes you want just one or two tools, not a whole counter. `from math import hypot, factorial, comb` pulls only those three names into your file, and after that line they can be called directly — `comb(10, 5)` — with no prefix at all.

This is the most convenient style when used sparingly, which is exactly why the pairing below is so instructive. The cell imports `seed`, `randint`, `choice`, and `sample` from `random`, then demonstrates the single most important idea in random programming: **seeding**.

Computers cannot produce true randomness by themselves — they compute, and computing is deterministic. `random.randint(1, 6)` doesn't roll a physical die; it reads the next value from a long mathematical sequence that *looks* random. That sequence is controlled by a **seed**: a starting number.

- Set `seed(42)` and the sequence begins at a specific point.
- Ask for three dice rolls and you get `[6, 1, 1]`.
- Call `seed(42)` again, and the sequence restarts at exactly the same point.
- Ask for three dice rolls again and you get the *same* `[6, 1, 1]`.

This is the superpower hiding inside "randomness": with a fixed seed, an experiment becomes reproducible. A failing test — or a paper you want to defend — can be re-run with identical results. `seed()` plus the `random` family turns lucky guessing into controlled science.

Also note `comb(10, 5) = 252`: the number of ways to choose 5 items out of 10, a preview of the "how many possible papers?" idea in the task notebook.

PYTHON

```
# 🐞 STYLE-3: `from module import name` — target-dish direct!
from math import hypot, factorial, comb
from random import seed, randint, choice, sample

print("booth-combo:", comb(10, 5), "ways")
seed(42) # 🐞 same-seed = same-dice!
print("dice x3   :", [randint(1, 6) for _ in range(3)])
seed(42) # re-seed: same-trick repeat!
print("redice x3  :", [randint(1, 6) for _ in range(3)], "← REPEAT ka proof!")
# Expected OUTPUT:
# booth-combo: 252 ways
# dice x3      : [6, 1, 1]
# redice x3    : [6, 1, 1] ← REPEAT ka proof!
```

1.4 Style 4 — `from module import *`: The Wildcard Trap

`from math import *` sounds incredibly convenient: "give me everything, no prefix, no fuss." It is — until it isn't. The star import takes *every* public name in the module and dumps it directly into your namespace. You don't get to choose; you get the whole counter thrown onto your plate.

The demonstration in this cell exposes the exact failure mode. Start with a plain user variable:

`ceil = 99` — your own, perfectly reasonable integer.

If, at that point, you had executed `from math import *`, Python's `math.ceil` function would have stomped over your variable. Conversely, if you already have your own `sum`, `min`, or `range`, they would stomp over the library's. Whichever assignment happens last wins, and nobody gets a warning. That silent, order-dependent overwriting is called **namespace pollution**.

The prefix style `math.ceil(9.2)` is immune: your `ceil` and math's `ceil` live in different namespaces, so both can exist peacefully. The quote in this cell sums it up: *prefix = the trustworthy road*.

The markdown cell right after this demo lists the three concrete harms — pollution, origin confusion, and linter blindness. When you import 200+ names you can no longer tell where any of them came from, and tools that catch typos and shadowing go quiet. The professional rule of thumb: prefer `import math` or an explicit `from math import ceil`; reserve the star for quick interactive experiments at your own risk.

PYTHON

```
# ⚠️ STYLE-4: `from module import *` – sab-kuch NAKED-import ka big-bill!
# (demo WITHOUT-star: cool-concept explain; star-import pollution!)
print("ceil ke do-chehre – same-naam ke saamne:")
# Scenario: star-import ne `ceil` ko math.ceil se OVERWRITE!
import math
ceil = 99 # user-variable
print("user-ceil :", ceil)
# agar `from math import *` kar diya hota:
# → math.ceil ne 'ceil' ko crush ho-jaata boom! (bhaari bill!)
print("math.ceil :", math.ceil(9.2), "(prefix = bharosa raasta!)")
# Expected OUTPUT:
# ceil ke do-chehre – manas ke saamne:
# user-ceil : 99
# math.ceil : 10 (prefix = bharosa raasta!)
```

The Harm of `import *` (3-part warning!)

1. **Namespace pollution:** 200+ names mix into your pocket — risk of your variables being hijacked (`sum` / `min` get taken over!)
2. **Origin confusion:** where did this function come from? (maintenance nightmare 🤪)
3. **Lint pain:** IDEs/linters cannot track silent imports! > 📌 **Rule-card:** for script clarity use `import X` or `from X import ...`

2. Inspecting Modules

2.1 Peeking Inside with `dir()` and the Dunders

Modules can feel like black boxes until you learn to open them. Python gives you three instant inspection tools, all used in this cell:

- `type(math).__name__` — the class of the object, which prints `"module"`. This confirms that a module is just another kind of Python object.
- `math.__name__` — the module's own name, `"math"`. Every module knows who it is.
- `dir(math)` — the complete inventory of names inside the module. Filtering out names starting with an underscore (`_`) removes the private machinery and leaves the *public API*: the functions the library authors intend you to call.

On this machine, `dir(math)` reveals more than sixty public tools. The letters `acos`, `cbrt`, `comb`, `log`, `sin`, and `tan` are all there — the entire counter, listed for free. In an interactive console, running `dir(math)` yourself is often faster than searching the documentation, because the answer is right there in the interpreter.

The double-underscore names you see when you *don't* filter are called **dunders** (double underscores). Attributes like `__doc__`, `__file__`, and `__name__` are Python's metadata layer: they describe the module itself. Good libraries also give every function a `__doc__` string, which lets you request `help(math.sqrt)` and read a plain-English description. If you write your own modules with docstrings (the next lesson), you are giving your teammates exactly the same luxury — a library that documents itself.

PYTHON

```
# Module peeping: `dir()` + module-name/type!
import math, random, datetime
print("math-module type:", type(math).__name__)
print("math.__name__ :", math.__name__)

peek = [n for n in dir(math) if not n.startswith("_")]
print("math's public-tools (first-10):", peek[:10])
print("total public :", len(peek))
# Expected OUTPUT:
# math-module type: module
# math.__name__ : math
# math's public-tools (first-10): ['acos', 'acosh', 'asin', 'asinh', 'atan', 'atan2', 'atanh', 'cbrt', 'ce
# total public : (varies ~50+, signature version-stable head!)
```

3. The Shadow Trap — Never Name a File Like the Standard Library

3.1 When Your Own `math.py` Silently Replaces the Real One

Here is an interview classic that has burned thousands of beginners — and a few professionals. Imagine you create a small helper file and, in a hurry, name it `math.py`. Somewhere else in the same project you write `import math`, expecting the square-root machine you have used all along.

Python looks for `math` by scanning its import paths. The directory containing *your* file is almost always on that search path — frequently before the standard library. So `import math` binds to **your** `math.py`, not the real one. The standard library is effectively hidden behind your file.

This is the **shadow trap**: a local module that shadows (overwrites) a standard-library name. Now every call to `math.sqrt`, every alias, and every learning snippet that depends on the real `math` silently breaks — or worse, `from math import sqrt` imports your file's `sqrt`, which may not exist at all.

The fix is absurdly simple: choose a different filename. The hall of shame names the usual suspects — `math.py`, `random.py`, `datetime.py`, `json.py`, `os.py`, `types.py`, `email.py`, `test.py`. If your file is on that list (or could be mistaken for any stdlib module), rename it before you import anything.

The complete live proof appears in the task notebook: a temporary fake `math.py` is created, `sys.path` is pointed at it first, and `import math` really does pick up the fake. After cleanup, the genuine standard-library `math` comes back. Seeing it happen once is worth a thousand warnings.

Summary

A module is a reusable `.py` file, and the standard library is a collection of such modules that ships free with Python. You can import any of them in four ways: `import X` keeps a clean, prefixed namespace; `import X as y` gives a short alias; `from X import name` pulls in only the tools you need; and `from X import *` dumps in every name and risks silently overwriting your variables. We used `math` for number drills, `random` with a fixed `seed(42)` to prove deterministic replay, and `datetime` to subtract dates and add timedeltas. Inspecting a module with `dir()` and reading its dunder names such as `__name__` and `__doc__` gives you a free tour of its public API. Finally, the shadow trap warns us never to name our own file `math.py` (or any stdlib name) — a mistake that silently hides the real library and breaks every dependent import.

You are no longer only a user of modules — you are their author. A module is simply a `.py` file of functions, classes, and variables, and importing it is a 3-step journey: check `sys.modules` for a cached copy, scan `sys.path` for the file, then execute it once and bind its namespace. We built and imported `grade_utils.py` live, peered inside with `dir()` and docstrings, and proved that a second import is cheap because the cache reuses the same object. The star of the show was the `__name__ == "__main__"` guard: it gives one file two lives — a silent library when imported, a loud script when run directly. Finally, clean `sys.path` and `sys.modules` after demos, and never recreate standard-library names like `math.py`. Next stop: packages — folders of modules with an `__init__.py` heart. 📦

A package is a folder of related modules marked by an `__init__.py` — the index-card that executes on first import, prints a banner, and re-exports shortcuts such as `from .billing import total_bill`. Packages can be imported three ways: the re-export shortcut (`dukan.total_bill(1000)`), the dotted submodule import (`from dukan.billing import tax`), and the whole-module import (`from dukan import catalog`). Submodules connect internally with relative imports (leading dots), which only work when the file is loaded as part of its package — so entry files live in the root folder, never run as `python dukan/billing.py`. Finally, the dunder quartet `__name__`, `__doc__`, `__package__`, and `__path__` (the true package marker) describe a package's identity, and `__pycache__` stores compiled bytecode. You are now ready to build your own package in the task notebook.

Virtual environments isolate every project's dependencies so conflicting library versions never fight. A venv is a self-contained folder with its own `python`, `pip`, and libraries in `bin/` (or `Scripts/` on Windows), `lib/`, and `pyvenv.cfg`. The 3-step mantra: create (`python -m venv myenv`), activate (`source myenv/bin/activate`; on Windows `myenv\Scripts\activate`), and leave (`deactivate`). Isolation is provable — inside the env, `sys.prefix != sys.base_prefix` prints `True`, and a fresh `pip list` shows an almost-empty shelf. The project passport is `requirements.txt`: export with `pip freeze > requirements.txt`, revive on another machine with `pip install -r requirements.txt`. Golden rules: keep env folders out of git (the recipe `requirements.txt` belongs in the repo), pip-install only after activating, and point your editor's interpreter at the env. Next stop: conda and uv — two more environment power tools.

This notebook mapped the three package power tools. **pip + venv** is the universal default — built into Python, reliable, light; its workflow is create → activate → `pip install` → `pip freeze > requirements.txt` → `deactivate`. **conda** manages environments AND non-Python packages (CUDA, ffmpeg, R libraries), uses `conda create -n ds python=3.11`, and exports to `environment.yml` — the data-science SUV. **uv** is the Rust-based bullet train: 10–100× faster installs, instant `uv venv`, and a modern `pyproject.toml` + lock-file workflow. The tool-pick matrix turns decisions into scenarios: learning/legacy → pip+venv; GPU/CUDA → conda; CI speed and modern monorepos → uv. Finally, the quick-answer drills rehearsed the mapping. Your default wisdom: master pip+venv first, then graduate to the right tool for the job. Next station — file handling and serialization with `json`, `csv`, and `pickle`.

Making Your Own Module — The First Box of Your Own Library

Real-World Hook: One Recipe Book for Every Kitchen

After two launched projects, you notice a pattern: you are writing the same helper functions again and again — averaging marks, mapping scores to grades, picking the topper. Photocopying those helpers into every new notebook works for a while, but every bugfix now has to be repeated in every copy.

The smart idea: one **recipe book** — a single `grade_utils.py` that every counter imports. Fix the recipe once; every kitchen benefits. That is the entire philosophy of modules: write code once, reuse it with an import, treat your own file with the same respect you give the standard library.

Definitions (interview-grade)

module — any `.py` file that can be imported. **import cycle** — 1) look in the `sys.modules` cache; 2) scan the folders in `sys.path`; 3) found → execute the file → bind the namespace.

The first cell below walks you through this exact cycle in a few lines of code.

1. Your First Module — Write, Point, Import

1.1 A module is just a `.py` file

There is nothing magical about a module. It is a plain Python file that defines functions, classes, and variables at the top level. When you run `import grade_utils`, the interpreter looks for a file called `grade_utils.py` in the folders listed in `sys.path`. If it is found, Python executes the file once, collects every top-level name into a module object, and binds that object to the name `grade_utils`. If the file is not found, you get `ModuleNotFoundError`.

The demo writes a real module with `open()`. It contains three functions: `average(marks_list)` computes `round(sum / len, 1)`, `grade_letter(score)` maps numbers to A/B/C/D/F through `if` thresholds, and `topper(students)` returns the name with the highest score using `max(students, key=students.get)`. That `key=` trick is worth remembering: `max` compares dictionary *values* when you give it the `get` method as the key function.

Then comes the punctuation mark: `sys.path.insert(0, tmp)`. The temp folder now leads Python's search list, so the freshly written file becomes importable. The alias `import grade_utils as gu` shortens the namespace, and now every function is one dot away: `gu.average(...)`, `gu.grade_letter(...)`, `gu.topper(...)`.

SQL

```
# 🚀 STEP-1: Apna grade_utils.py likho + import karo! (tempdir-magic-live 🚀)
import sys, os, tempfile

MODULE_CODE = '''"""grade_utils – Student-marks ke helper-functions ki library! 📖"""

def average(marks_list):
    """Marks-list ka average nikaalo (round-1!)."""
    return round(sum(marks_list) / len(marks_list), 1)

def grade_letter(score):
    """90:A, 75:B, 60:C, 40:D, <40:F – exam-card style!"""
    if score >= 90: return "A"
    if score >= 75: return "B"
    if score >= 60: return "C"
    if score >= 40: return "D"
    return "F"

def topper(students):
    """{'name': marks} dict me se sabse-badiya-student name!"""
    return max(students, key=students.get)
'''

tmp = tempfile.mkdtemp()
with open(os.path.join(tmp, "grade_utils.py"), "w") as f:
    f.write(MODULE_CODE)

sys.path.insert(0, tmp) # python ko BATO: "yahan bhi dekh!" 📖
import grade_utils as gu # 🚀 private-library live!
# ...

# Expected OUTPUT:
# average: 80.0
# grade 88: B
# topper : Priya
```

1.2 Module peeping — `dir()` and docstrings

A module should be able to explain itself. The built-in `dir(module)` lists every name the module exposes — functions, variables, and also dunder machinery like `__name__` and `__doc__`. The demo filters those out with `[n for n in dir(gu) if not n.startswith("_")]`, leaving exactly the public surface: `['average', 'grade_letter', 'topper']`. That is a perfect sanity check: the module shows only what we designed it to show.

The second self-explanatory tool is the **docstring**. The triple-quoted comment at the top of the file becomes `gu.__doc__` — the module's own one-line biography. Each function's docstring is equally queryable: `gu.average.__doc__` prints what `average` promises to do. In an interactive console, `help(gu.average)` renders the signature and docstring into a tidy card.

Why does this matter at scale? Because documentation that lives next to the code cannot rot: it is regenerated from the file itself. When you revisit a module months later, `__doc__` and `dir()` give you an instant, accurate map.

PYTHON

```
# 🐡 STEP-2: Module-peeping – dir() + docstring-self-guide! 📖
peek = [n for n in dir(gu) if not n.startswith("_")]
print("public-tools:", peek)
print("doc      :", gu.__doc__)

# tip: console me help(gu.average) likho – full-card milega! 💡
func_doc = gu.average.__doc__
print("average-doc :", func_doc)
# Expected OUTPUT:
# public-tools: ['average', 'grade_letter', 'topper']
# doc      :    grade_utils – Student-marks ke helper-functions ki library! 📖
# average-doc :    Marks-list ka average nikaalo (round-1!).
```

1.3 The import cache — `sys.modules`

Run the same import twice and you might expect the file to execute twice. It does not. Python keeps every loaded module in a dictionary called `sys.modules`, keyed by the module's name. The first `import grade_utils` executes the file and stores the result. A second `import grade_utils as gu_2` finds the name already cached and returns the *same* object without re-reading the file — hence `gu is gu_2` prints `True`.

This cache is a performance feature: importing a heavy module (imagine hundreds of functions) would otherwise be catastrophically slow. But it is also the classic footgun in notebooks: after you edit your `.py` file on disk, re-importing still gives you the stale cached version. The official cure is `importlib.reload(module)`, or simply restarting the kernel. The rule to internalise: *a module executes exactly once per process* — the cache is your friend until you are actively editing, and then it feels like an enemy.

PYTHON

```
# 🐡 STEP-3: sys.modules cache – "already-load hai ji!" proof!
print("in-cache?  :", "grade_utils" in sys.modules)

# dobara-import: NAYA-execute NAHI – cache-se le-te! (speed-priya!)
import grade_utils as gu_2
print("same-object? :", gu is gu_2)
print("(cache-rule → import 2nd-time CHEAP; test-fixes require reload!)")
# Expected OUTPUT:
# in-cache?   : True
# same-object? : True
# (cache-rule → import 2nd-time CHEAP; test-fixes require reload!)
```

2. The Two Lives of a File

2.1 Import-silent vs script-loud

Every Python file has two possible lives. If you run `python myfile.py` in a terminal, the file is a **script**: its top-to-bottom code executes, `prints` fire, and it behaves like a program. If you `import myfile` from elsewhere, the file is a **module**: functions and classes load, but you usually do not want its demo prints and test blocks cluttering your session.

How does Python know which life a file is living? It sets a magic variable, `__name__`:

- when the file is executed directly → `__name__ == "__main__"` 🎬
- when the file is imported → `__name__ == "file's own name"` (e.g. `"loud_fetcher"`)

The idiom `if __name__ == "__main__":` therefore arms a block that fires only in script mode. The next cell builds a module with an unguarded loud print ("loud-banner-at-import") and a guarded print ("script-mode special-lines"). On import, only the banner fires; the guarded line stays silent — exactly the professional behaviour you want from a library.

SQL

```
# 🔑 STEP-4: __name__ == "__main__" guard 🚫 TVN-democracy-scene!
# Scene: module ke andar loud-prints 📢
LOUD_MODULE = '''DB = {"sana": 90}

def fetch(name):
    return DB[name]

print("loud-banner-at-import 📢")

if __name__ == "__main__":
    print("script-mode special-lines! 🎬")
'''

tmp2 = tempfile.mkdtemp()
script_path = os.path.join(tmp2, "loud_fetcher.py")
with open(script_path, "w") as f:
    f.write(LLOUD_MODULE)

sys.path.insert(0, tmp2)
print("=== IMPORT-side ===")
import loud_fetcher # sirf print-1 chalega!
print("fetch-result:", loud_fetcher.fetch("sana"))
# Expected OUTPUT:
# === IMPORT-side ===
# loud-banner-at-import 📢
# fetch-result: 90
```

2.2 Running the same file as a script — subprocess

The previous cell proved what *import-time* silence looks like. This cell proves the sunny side: the same file, launched directly, becomes chatty. To launch it, we need a brand-new Python process — not the notebook's kernel — so the demo uses `subprocess.run([sys.executable, script_path], capture_output=True, text=True)`.

`sys.executable` is the full path of the Python interpreter currently running the notebook — passing it guarantees we use the very same Python. `capture_output=True` captures the child's stdout into `result.stdout`, and `text=True` decodes it as text (instead of bytes). `result.stdout.strip()` then gives us exactly what a terminal user would have seen: both the banner AND "script-mode special-lines!", because in script mode `__name__` is `"__main__"`.

The contrast between the two cells is the whole lesson: **the guard makes one file behave as a quiet library when imported and as a loud program when executed.** Finally, the cleanup (`sys.path.pop(0)`) twice, plus evicting both modules from (`sys.modules`) restores the environment so later cells start fresh.

PYTHON

```
# 📁 STEP-5: SAME-file ko SCRIPT-mode run (subprocess!) – guard-line fires!
import subprocess
result = subprocess.run([sys.executable, script_path],
                        capture_output=True, text=True)
print("=== SCRIPT-side ===")
print(result.stdout.strip())
print("(notice: dono-prints yahan! – import-side pe sirf-eka-PRINT!) 🔑")

# cleanup-ka pack
sys.path.pop(0); sys.path.pop(0)
sys.modules.pop("grade_utils", None); sys.modules.pop("loud_fetcher", None)
# Expected OUTPUT:
# === SCRIPT-side ===
# Loud-banner-at-import 📢
# script-mode special-lines! 📁
# (notice: dono-prints yahan! – import-side pe sirf-eka-PRINT!) 🔑
```

The Meaning of the Guard (the golden rule)

- running `python file.py` → Python sets `__name__` to the special string `"__main__"` 📁
- running `import file` → Python sets `__name__` to the module's name, `"file"` (silent import!)

Rule to write down: put demo code, self-tests, and “run-me” examples inside the guard block. That way your module stays quiet and polite when somebody imports it, yet fully alive when launched directly. Every serious Python file you will read in the wild obeys this rule.

PYTHON

```
if __name__ == "__main__":
    # these lines run ONLY when the file is executed directly
    # on the import side they are skipped – the module stays a silent library
```

Packages Structure — The Family Tree of Files in the Cupboard

Real-World Hook: Order in the House-Cupboard

Everything mixed in one drawer becomes a hotchpotch you can never find anything in. The organised alternative is **labelled boxes**: each box holds related items, and the outside of the box says what is inside.

In Python, a folder becomes a package when it contains `__init__.py` — the index-card. That file can run setup code, print a banner, and re-export shortcuts so the family's most-used names are one step away.

📖 Definition (interview-grade)

Package — a folder containing `__init__.py` and its submodules, importable by dotted name (`dukan.billing`). On the first import, `__init__.py` **executes**, doing the wiring and re-export work. (Modern Python also accepts a “namespace package” without `__init__.py` — but the explicit classic style is clearer and what the ecosystem mostly uses.)

PYTHON

```
home/
├── kitchen/  ← a package! ( __init__.py = the index-card )
│   ├── daal_shelf.py  ← a submodule
│   └── masala_rack.py  ← a submodule
└── study/  ← a package!
    └── notes.py
```

1. The Package Build — A Folder, an `__init__.py`, and Three Submodules

1.1 Making a package with bare hands

The next cell assembles a complete package from nothing: an empty temp folder, then three files written with `open()`. The folder is named `dukan` (a little corner shop), and inside it sit the classic trio:

- `__init__.py` — the package's identity card. It carries the docstring, prints the welcome banner (“dukan-package banner: welcome ji! 🙏”), and performs the re-export `from .billing import total_bill`. That last line is the magic shortcut: after `import dukan`, callers can say `dukan.total_bill(...)` instead of digging into `dukan.billing.total_bill(...)`.
- `billing.py` — the money submodule. It defines a `TAX_RATE = 0.05` constant, a `tax(amount)` rounding 5% to two decimals, and `total_bill(amount)` returning amount plus tax.
- `catalog.py` — the price-board submodule. `ITEMS` maps dish names to prices, and `describe(item)` formats a dish as “name ₹price”.

1.2 The import mechanics

The parent folder — the one *containing* `dukan` — is inserted into `sys.path`, not the package itself. That is because a package is found by name: Python looks for a *folder* called `dukan` in one of the `sys.path` entries. The tree-print is just our own documentation of what was built, and then the real event happens: `import dukan` executes its `__init__.py`, and the banner rings out — the package has loaded.

PYTHON

```
# 🐘 STEP-1: Dukaan-package banao + import karo! (tempdir-full-tree! 📁)
import sys, os, tempfile

BASE = tempfile.mkdtemp()
PKG = os.path.join(BASE, "dukan")
os.mkdir(PKG)

INIT_CODE = '''"""dukan - gali-ka-store family-package! 🏠"""
print("dukan-package banner: welcome ji! 🙋")

from .billing import total_bill      # SHORTCUT re-export! 🏠
'''

BILLING_CODE = '''"""billing - tax+totals ka hisaab! 💰"""

TAX_RATE = 0.05

def tax(amount):
    """5% tax nikaalo amount pe!"""
    return round(amount * TAX_RATE, 2)

def total_bill(amount):
    """amount + tax ka grand-total!"""
    return round(amount + tax(amount), 2)
'''

CATALOG_CODE = '''"""catalog - items ke price-board! 📋"""

# ...
# |— __init__.py   (banner + re-export 🏠)
# |— billing.py    (tax/total_bill 💰)
# |— catalog.py    (ITEMS + describe 📋)
# dukan-package banner: welcome ji! 🙋'''
```

1.3 Three import styles for packages

Packages multiply the import options you already know:

Style A — package-level re-export. Because `__init__.py` did `from .billing import total_bill`, the name `total_bill` became part of the package's top level. `dukan.total_bill(1000)` → `1050.0` (1000 + 5% tax) works with no submodule import at all. This is why `__init__.py` is called the shortcut-maker.

Style B — dotted submodule import. `from dukan.billing import tax` descends folder → file → name, then you call bare `tax(2000)` → `100.0`. The dotted path is explicit about *where* the function lives — great for readability and for linters to trace.

Style C — module import. `from dukan import catalog` grabs the whole submodule as a name, so you use `catalog.describe("samosa")` (→ `samosa ₹20`) and `catalog.ITEMS["jalebi"]` (→ `80`). This keeps related functions grouped under one object.

All three are the same data behind different doors. Choosing one is about legibility: shortcuts for hot public APIs, dotted imports for deep access, module imports when you want everything a submodule offers.

PYTHON

```
# 📄 STEP-2: 3-import-styles for packages!
# ORDER-PATTERN-A: package-re-export shortcut call!
print("via __init__ :", dukan.total_bill(1000)) # from .billing re-export!

# ORDER-PATTERN-B: dotted-import submodule!
from dukan.billing import tax
print("dotted-import :", tax(2000), "(5% of 2000!)")

# ORDER-PATTERN-C: from-package-import-module!
from dukan import catalog
print("module-import :", catalog.describe("samosa"))
print("catalog-price :", catalog.ITEMS["jalebi"])
# Expected OUTPUT:
# via __init__ : 1050.0
# dotted-import : 100.0 (5% of 2000!)
# module-import : samosa ₹20
# catalog-price : 80
```

2. Relative Imports — Wiring Inside the Package

2.1 Dots mean “inside this package”

Submodules often need each other. How does `billing.py` reach the price board in `catalog.py`? Through a **relative import**: `from .catalog import ITEMS`. The single leading dot means “the package I live in”, so the path is resolved relative to the current module's package, not the project root.

That is what `__init__.py` did with `from .billing import ...` — the dot says “billing, my sibling, in my package dukan.” Relative imports keep package wiring portable: you can rename the outer project folder or move the whole package, and the internal connectors still work, because they are relative to the package, not absolute.

2.2 The one sharp rule: never run a submodule directly

Relative imports work only when the file is loaded *as part of its package* — i.e. reached through an import like `import dukan` or `from dukan import billing`. If you try `python dukan/billing.py`, Python treats `billing.py` as a standalone script with no enclosing package, the leading dot has nothing to resolve, and you get `ImportError: attempted relative import with no known parent package`.

The practical consequence is the golden layout: **entry files (the ones you run) sit in the root folder; submodules are only imported, never executed directly**. The cell below states this as a set of rules to carry into your own projects.

PYTHON

```
# 🚩 STEP-3: Relative-imports – package-internal wiring ka note! 📌
# billing.py ne catalog-se-something chahiye hota toh:
#     from .catalog import ITEMS      ← SAME-package sibling!
# Dot-ka-matlab: "current-package inside". Direct-run pe relative FAIL:
#     python billing.py → ImportError: attempted relative import...
# > packages ke subs script-run-mat-karo; entry-file root-folder me rakho!

print("RELATIVE-IMPORT GYAAN:")
print("  billing.py → `from .catalog import ITEMS` ✅ (sibling-link!)")
print("  root-run   → `python dukan/billing.py` ❌ (relative-na-mile!)")
print("  entry-file → root folder me `main.py` rakho ✅")
# Expected OUTPUT:
# RELATIVE-IMPORT GYAAN:
#  billing.py → `from .catalog import ITEMS` ✅ (sibling-link!)
#  root-run   → `python dukan/billing.py` ❌ (relative-na-mile!)
#  entry-file → root folder me `main.py` rakho ✅
```

3. The Dunder Zoo — How a Package Describes Itself

3.1 Reading a package's identity card

The last live demo asks `dukan` to reveal itself through its **dunder attributes** — the double-underscore metadata every module and package carries:

- `dukan.__name__` → `"dukan"` — the package's registered import name.
- `dukan.__doc__` → the docstring written in `__init__.py` — its one-line biography.
- `dukan.__package__` → `"dukan"` — the package a module belongs to (a top-level package is its own parent).
- `dukan.__path__` → the list of directories where Python will hunt for submodules. This is the defining trait of a package: *only* packages have `__path__`; plain modules do not. `len(dukan.__path__) == 1` prints `True`, confirming exactly one folder backs this package.

These four attributes are the fastest way to interview an unfamiliar package: who are you, what do you promise, and where do your children live? Combined with the earlier `__pycache__` mention, you now see the whole lifecycle: source files → bytecode cache → loaded package object → queryable dunder. The final cleanup (`sys.path.pop(0)` and `sys.modules.pop("dukan", None)`) restores the import system for the next notebook.

PYTHON

```
# 📦 STEP-4: Dunder-zoo peep (package-vals!) 🔍
print("dukan.__name__ :", dukan.__name__)
print("dukan.__doc__ :", dukan.__doc__)
print("dukan.__package__:", dukan.__package__)
print("dukan.__path__ : [folder-check]", len(dukan.__path__) == 1)

# cache-clean-tyag (demo KEEP-safe!)
sys.path.pop(0); sys.modules.pop("dukan", None)
# expected-na:
# Expected OUTPUT:
# dukan.__name__ : dukan
# dukan.__doc__ : dukan - gali-ka-store family-package! 🍷
# dukan.__package__ : dukan
# dukan.__path__ : [folder-check] True
```

Virtual Environments — Your Own Kitchen for Every Project

Real-World Hook: The Kitchen-War of 2 Projects

Project-A (a pizza app 🍕) needs `cheese_lib` v1; Project-B (a salad dash 🥗) needs `cheese_lib` v2 with the same name. In ONE global microwave (global pip!) only one can be installed — a nasty tug-of-war: whoever installs last is the winner, and the other project breaks.

The root solution: a **separate kitchen** for every project. Separate counters, separate lib-versions, separate pip-life — no war. A virtual environment is exactly that kitchen: an isolated folder holding its own `python`, its own `pip`, and its own libraries, completely locked away from the base system.

📖 Definition (interview-grade)

Virtual Environment = an isolated folder that owns its own `python`, its own `pip`, and its own installed libraries, fully separated from the system-level installation. `activate` = the entry pass 🚪 that makes your shell use that folder's tools; `deactivate` = leaving the kitchen.

The live demo next will build one of these kitchens and inspect its rooms.

1. Creating a Kitchen — `venv` Anatomy

The `venv` concept is simple; the folder it creates is a small, organised world. The next cell runs `python -m venv <folder>` and then inspects what was built.

1.1 What `python -m venv` creates

`python -m venv myenv` builds a new self-contained Python world inside the folder `myenv`. Inside it you will typically find:

ENTRY	PURPOSE
<code>bin/</code> (Linux/macOS) or <code>Scripts/</code> (Windows)	launcher scripts: your own <code>python</code> , <code>pip</code> , plus the activation scripts (<code>activate</code> , <code>Activate.ps1</code> , <code>activate.csh</code>)
<code>lib/</code> (or <code>Lib/</code>)	the private site-packages where your env's libraries get installed — isolated from the global site-packages
<code>include/</code>	C-extension headers used when building packages with native code
<code>pyvenv.cfg</code>	the config card pointing back to the base (system) Python that spawned this env
<code>.gitignore</code>	a polite helper so you do not accidentally commit the heavy env folder (newer versions add it automatically)

The demos use a temp directory for the environment, but in a real project you would create `myenv` right inside the project folder. The activation scripts in `bin/` are the magic behind the `(myenv)` prompt you will see in terminals — that prompt is the signal that your shell commands now run inside this kitchen.

The expected output in the cell shows exactly this layout (`bin`, `include`, `lib`, `pyvenv.cfg` — plus `.gitignore` on this Python version), and a peek into `bin/` reveals the launcher and activation files. That is the whole anatomy: interpreters, launchers, config, and an isolated library shelf.

```

PYTHON

# 🍹 STEP-1: venv ka anatomy (folder structure-preview!) 📁
import sys, os, subprocess, tempfile

BASE = tempfile.mkdtemp()
ENV = os.path.join(BASE, "demov")
print("creating venv at:", ENV, "(takes ~10-20s...)")
subprocess.run([sys.executable, "-m", "venv", ENV], check=True)
print("\n📁 venv-internal folders:", sorted(os.listdir(ENV)))
print("🔍 venv/bin sneak-peek :", sorted(os.listdir(os.path.join(ENV, "bin"))[:5]))
# Expected OUTPUT:
# creating venv at: /tmp/... (takes ~10-20s...)
#
# 📁 venv-internal folders: ['bin', 'include', 'lib', 'lib64', 'pyvenv.cfg']
# 🔍 venv/bin sneak-peek : ['Activate.ps1', 'activate', 'activate.csh', 'pip', 'pip3']

```

1.2 Proof of isolation — `sys.prefix` vs `sys.base_prefix`

How do we *prove* an environment is actually isolated rather than just a folder with a nice name? We ask the environment's own Python two questions and compare the answers.

- `sys.base_prefix` — the location of the base (system) Python that created the environment.
- `sys.prefix` — the location of the Python that is currently running.

Inside a virtual environment these two differ: `sys.prefix` points into your env folder while `sys.base_prefix` still points at the system Python. The check `sys.prefix != sys.base_prefix` therefore prints `True` — the mechanical proof that `demov/bin/python` is running inside its own isolated world. Outside an environment, in the base interpreter, both values are equal and the check prints `False`.

The cell also runs `pip list` from the env's own pip: a fresh environment contains almost nothing (just pip and setuptools on classic builds) — the clean slate that guarantees no global packages leak into your kitchen. Two numbers (`prefix` mismatch) and a short package list tell you everything: interpreters differ, libraries are empty — you are truly isolated.

The anatomy note after the demo reminds you how Windows mirrors everything under `Scripts/\activate` instead of `bin/activate`, and why keeping envs out of git is the industry rule.

PYTHON

```
# 🐍 STEP-2: ISOLATION-PROOF – venv-python hi apna-world! 🚩
pybin = os.path.join(ENV, "bin", "python")

ver = subprocess.run([pybin, "-V"], capture_output=True, text=True).stdout.strip()
print("venv-python :", ver)

proof = subprocess.run([pybin, "-c",
    "import sys; print(sys.prefix != sys.base_prefix)"],
    capture_output=True, text=True).stdout.strip()
print("isolated? 🚩:", proof, "(venv-prefix ≠ base-prefix!)")

# venv-pip list – tiny-fresh-start!
pl = subprocess.run([os.path.join(ENV, "bin", "pip"), "list", "--format=freeze"],
    capture_output=True, text=True).stdout.strip()
print("venv-pkgs   :", len(pl.splitlines()) if pl else 0, "packages (sirf pip/setuptools-fresh-start! 🌱)")
# Expected OUTPUT:
# venv-python : Python 3.13.14 (aapke-wala!)
# isolated? 🚩: True (venv-prefix ≠ base-prefix!)
# venv-pkgs   : 2 packages (sirf pip/setuptools-fresh-start! 🌱)
```

Anatomy-amusements:

- `bin/` → activate-scripts + your own `python` + `pip` (Linux/Mac) 🐼 (on Windows the folder = `Scripts/` + `activate.bat`! 📄)
- `lib/` → all the libraries of this kitchen (isolated-racks!)
- `pyenvv.cfg` → the config-card (the base-python linker!) > Notice Step-2's `True` — **venv-prefix ≠ base-prefix** = the isolation > formula! This is the THE-END pass for project-wars! 🙌

Terminal-CASTLE — The 3-Step Command Mantra!

🐼 Linux/Mac:

📄 Windows (cmd / PowerShell):

📄 Revive a project on a new machine (the Requirements passport!):

BASH

```
python3 -m venv myenv           # 1) kitchen-build (naye-folder!)
source myenv/bin/activate       # 2) ENTRY-pass 🚧 (prompt-start: (myenv))
python -V; which pip           # 3) INSIDE-proofs – venv-python vad-hi!
pip install numpy pandas       #   Libs isi-kitchen me!
pip freeze > requirements.txt   #   passport-export 📄
deactivate                     # 4) EXIT! (prompt normal!)
```

BASH

```
python -m venv myenv
myenv\Scripts\activate         :: cmd-style
myenv\Scripts\Activate.ps1     :: PowerShell-style (run-policy care!)
python -V
pip install jupyter
deactivate
```

Venv-usage rules — Golden-Rules!

RULE	WHY
1. venv-folder NOT in GIT (.gitignore!)	heavy folder + machine-specific!
2. keep only requirements.txt in the repository!	recipe-passport (small + portable!) 📄
3. your own env for every project	dependency-war THE-END 🙌
4. pip-install only after activate	otherwise global-pollution!
5. deactivate when you exit	prompt safe, world safe! 🚧

🐞 **Common-mistake:** tell your editor/IDE about the venv too (VSCode: Ctrl+Shift+P → "Python: Select Interpreter" → ./myenv/bin/python!) — otherwise the editor complains "module not found!" even though it was pip-installed! 🐦🐱

conda vs pip vs uv — Choosing Your Package Power Tools

Real-World Hook: The Evolution-Chart of 3 Transports

The notebook's big-picture definitions:

- **Environment-manager** = the tool that builds and enters isolated kitchens.
- **Package-installer** = the tool that fetches and installs libraries.
- pip, conda, and uv all play BOTH roles, which is why they get compared as complete solutions.

📖 Definition (interview-grade)

Environment manager — builds isolated environments (venvs). **Package installer** — downloads/installs packages into an environment. **pip + venv · conda · uv** — each of these tools can handle both jobs, so the real question is: which piece of machinery fits this project?

The machine-detective cell next tells you what is actually available on this system.

PYTHON

```

pip+venv = CYCLE 🚲 📦 reliable, found everywhere, light!
conda    = SUV 🚗 📦 heavy capacity; carries non-Python bags too!
uv       = BULLET-TRAIN ⚡ 📦 same city, but 10-100x faster (Rust magic!)
```

1. The Three Jugglers — pip, conda, uv

1.1 Machine detective: what's on this system?

The first live cell is a 10-second system checkup. It uses `shutil.which(name)` — a standard-library function that returns the full path to an executable if it is on the system PATH, or `None` if it is not. The pattern `shutil.which("conda") is not None` converts that into a clean boolean: `True` means “installed and callable”, `False` means “not here”.

The cell also prints the running Python version (`sys.version.split()[0]` → e.g. `3.13.14`) and counts globally installed packages via `python -m pip list --format=freeze`. That count is your warning shot: dozens or hundreds of shared packages in one global namespace is precisely the pollution venv/conda/uv prevent.

Run the cell and read the booleans — they are the environment's honest status report. Then the note that follows explains why `False` for conda/uv is completely normal and does not block anything in this lesson.

PYTHON

```
# 🐞 STEP-0: machine-detective – konsa-tool available? (sandbox-live!) 🐞
import shutil, subprocess, sys
print("pip available :", shutil.which("pip") is not None)
print("pip available (python-m):", shutil.which("pip3") is not None or True)
print("conda available:", shutil.which("conda") is not None, "(optional-install!)")
print("uv available :", shutil.which("uv") is not None, "(optional-install!)")
print("\npython      :", sys.version.split()[0])

# pip-list count (machine-life!)
pkgs = subprocess.run([sys.executable, "-m", "pip", "list", "--format=freeze"],
                      capture_output=True, text=True).stdout.strip()
n = len(pkgs.splitlines()) if pkgs else 0
print("global-pkgs :", n, "(machine-ka-count – yehi global-pollution ki misaal!)")
# Expected OUTPUT:
# pip available : True
# pip available (python-m): True
# conda available: False (optional-install!)
# uv available : False (optional-install!)
#
# python      : 3.13.14
# global-pkgs : ~57-60 (machine-ka-count – yehi global-pollution ki misaal!)
```

Important-moment: `uv` and `conda` are probably NOT installed — and that's OK!

The detective cell just printed the diagnosis: pip (and `python -m pip`) are present, while conda and uv returned `False`. That is the expected default — pip+venv comes **built-in** with every Python; conda and uv are optional superpowers you install separately when a project needs them. The count of global packages (often in the hundreds) is the living example of global pollution — exactly the problem environments solve. Treat conda and uv as tools to graduate into, not prerequisites. The rest of this notebook gives you the cheat-sheets so that when you do need them, you are ready.

pip+venv — The Universal Default Cheat-Sheet (recap-link!)

Pro-pins: `==` exact-pin · `>=` min-pin · `~=` compatible-pin (minor-bumps!)

BASH

```
python3 -m venv myenv      # create
source myenv/bin/activate  # enter 🚪
pip install numpy==2.1     # pin-versions (stability-locks!)
pip list                   # inventory 📋
pip show numpy             # detail-card (version+location!)
pip uninstall numpy        # remove
pip freeze > requirements.txt # passport-export! 🛂
pip install -r requirements.txt # revive on new-machine!
deactivate                 # exit 🚪
```

Conda — The DS-classic Cheat-Sheet! 🐼

Conda = envs-TOO + non-Python packages TOO (C++/R-libraries, CUDA, ffmpeg — the full-suv of wheels!!)
After Anaconda/Miniconda install:

Conda-vs-pip compare-table:

FEATURE	PIP(+ VENV)	CONDA
packages	Python-only (PyPI!)	Python + non-Python (C/CUDA...) 🚧
python-version-switch-env	complex (pyenv joker!)	easy via <code>python=3.11</code> arg ✅
speed	reliable-slow	slower-resolver (legacy!) 🐢
DS-enterprise scene	OK	CLASSIC-land (Jupyter+GPU friends!)
install-size	light	heavy (1-3GB anaconda weight!)

🎯 **When conda:** GPU/CUDA projects, R-mix teams, the "single-tool for every package" comfort — otherwise pip+venv nice-and-slim! 🐈

BASH

```
conda create -n ds python=3.11      # env-create (pin the python-version!)
conda activate ds                    # enter 🚪
conda install numpy pandas jupyter  # conda-forge style-friends!
conda install -c conda-forge ffmpeg # non-python swiss-sneaks too! 🚗
conda list                           # inventory 📋
conda env export > environment.yml   # passport-export (yaml!)
conda env create -f environment.yml   # revive!
conda deactivate                     # exit 🚪
```

uv — The Bullet-Train Cheat-Sheet! 🚄

uv = a modern toolset written in Rust — **the same pip-compatible workflow, but 10-100× faster** (parallel-downloads + smart-resolver + cache!) 🌸 (the Astral-team — the ruff-linter folks!)

Speed-feel (a typical work-life anecdote!):

🎯 **When uv:** teams with large-stacks, CI-speed, and a modern-workflow (pyproject+lock!) — bullet-train your developer-life! 🚄

BASH

```
# INSTALL once: pip install uv (ya: curl -LsSf astral.sh/uv/install.sh | sh)
uv venv                                # instant-env create (0.0x seconds!)
source .venv/bin/activate              # enter 🚪 (same-feel!)
uv pip install numpy pandas            # bullet-install ⚡ (same-syntax!)
uv pip freeze > requirements.txt        # passport-export!
uv init myproj                         # NEW-project scaffold + pyproject.toml 🌟
uv add numpy                           # pyproject-managed-add (lock-file swag!)
uv run script.py                       # auto-env-run (activation-free!) 🚄
uv lock --upgrade                      # lockfile refresh!
```

BASH

```

pip install big-stack → 🍷 chai banana shuru (1-3 min!)
uv pip install same → ⚡ blink-return (5-15 sec!) 🚀 🧠

```

TOOL-PICK MATRIX (scenario → the best buddy!)

SCENARIO	TOOL	WHY
learning/scripts 🧑	pip+venv	zero-setup, built-in 🌱
web-app/APIs 🌐	pip+venv (or uv!)	standard-ecosystem
data-science + GPU/CUDA 🧠	conda	non-python libs (CUDA ...)
R/C/C++ mixed-stack 🦄	conda (or pixi)	language-blend-deps
CI-fast-farms ⚙️	uv	cache+speed rocket 🚀
modern-monorepos/proj-manag	uv (pyproject+lock!)	reproducible-builds
legacy-school-projects 🏠	pip+venv	everywhere-works

pyproject.toml, the new modern-standard (brief!)

in modern-projects, alongside `requirements.txt` comes **pyproject.toml** (project-metadata + dep-list + build-settings!) — the uv/web-family home! ✨ detail: advanced-territory — keep the confidence, it will come in handy!

2. Decision-Time Drills**2.1 Quick-answer rehearsals**

The big cheat-sheets are reference material; this cell makes the decisions stick by rehearsing them in a tiny loop. The data structure is a list of `(question, answer)` tuples, and the loop prints each pair with `f"Q: {q: <30} → A: {a}"` — the `{q:<30}` format spec left-aligns the question inside a 30-character column, which is why every arrow lines up neatly.

Work through the five pairs as a mental checklist:

QUESTION	ANSWER YOU SHOULD REACH FOR
Minimal setup for any machine?	pip+venv — built-in
GPU + CUDA heavy stack?	conda — non-Python binary packages
CI pipeline speed?	uv — Rust-speed resolve
pyproject + lock-file style?	uv/pdm/poetry modern family
Anaconda's weight?	~1–3 GB full-suite

The drill is deliberately self-contained — run it, re-read the table, and the scenario→tool mapping starts to become second nature.

PYTHON

```
#  QUICK-ANSWER DRILLS (interactive-feel! )
drills = [
    ("Har-machine minimal?", "pip+venv – built-in!"),
    ("GPU + CUDA setup?", "conda (non-python stack!)"),
    ("CI speed rocket?", "uv – Rust-speed!"),
    ("pyproject+lock-style?", "uv/pdm/poetry modern-family!"),
    ("anaconda ka weight?", "~1-3GB (full-suite!)"),
]
for q, a in drills:
    print(f"Q: {q:<30} → A: {a}")
# Expected OUTPUT:
# Q: Har-machine minimal?           → A: pip+venv – built-in!
# Q: GPU + CUDA setup?             → A: conda (non-python stack!)
# Q: CI speed rocket?              → A: uv – Rust-speed!
# Q: pyproject+lock-style?         → A: uv/pdm/poetry modern-family!
# Q: anaconda ka weight?           → A: ~1-3GB (full-suite!)
```

Common Mistakes and Pitfalls

5 COMMON MISTAKES

#	✗ MISTAKE	✓ CORRECT
1	<code>from math import *</code> again and again	prefix-style <code>import math</code> for safety
2	your filename = <code>stdlib (math.py)</code> shadow-trap	give your files a different name! 🗑️
3	<code>from random import *</code> + your own <code>sample()</code> clash	use only from-name-import (explicit!)
4	mixing string-dates in <code>datetime.date()</code>	parse the text → <code>datetime.strptime</code>
5	thinking seed = "always lucky"	seed is only a deterministic repeat (testing tool!) 🎲

🎯 **Toolbox-card:** math=numbers-pump 📖 · random=seeded-luck 🎲 · datetime=calendar-train 🚂
 🛠️ — install-free, docs-rich, import just this!

#	MISTAKE	CORRECT HABIT
1	unguarded loud prints in a module file	protect with <code>if __name__ == "__main__":</code> 🔑
2	leaving temp folders crammed into <code>sys.path</code>	after the demo pop them out + evict from <code>sys.modules</code> 🗑️
3	"I edited my file but the old result keeps coming back!"	the import cache! use <code>importlib.reload(module)</code> 🔄
4	scattering imports deep inside functions	keep imports at the top of the file (PEP 8) — unless genuinely needed at runtime
5	filenames with spaces, dashes, or capitals	lowercase <code>snake_case</code> is the Python way 📁

Cache principle: importing the same module a second time is nearly free (the cache serves it); the first import executes the whole file. The guard is what keeps that one-time execution from being noisy.

#	MISTAKE	CORRECT HABIT
1	forgetting <code>__init__.py</code> (a plain folder, not a package!)	add <code>__init__.py</code> — even a docstring-only one is enough 📁
2	heavy compute inside <code>__init__.py</code> (slow import!)	keep init light: shortcuts and banners, not business logic 🍷
3	running a submodule directly: <code>python dukan/billing.py</code>	keep entry files at the root; let the package be imported from above 🚫
4	<code>from dukan import *</code> — pollution travels to packages too	use explicit names or dotted imports
5	package names in CAPITALS or with dashes	lowercase <code>snake_case</code> is the ecosystem standard 📁

🔗 **Import-resolution map:** cache → path-scan → folder-peep → `__init__`-execute → dotted-binds. Understanding each step makes any package mystery solvable in minutes.

#	❌ MISTAKE	✅ CORRECT
1	installing EVERYTHING global-pip on one machine	venv-per-project (war-avoid!) 🙌
2	randomly mixing conda+pip in the same-env	first conda-big, then pip-rest — COLD-ORDER!
3	tearing up requirements.txt without a freeze	on every change <code>pip freeze > ...</code> sync! 📄
4	expecting uv everywhere (WSL/termux issues sometimes!)	pip+venv as the reliable-fallback 🌱
5	tool-hoarding (setting everything up at once!)	need-driven pick (matrix-rule!) 📖

🔗 **Default-Wisdom:** learn pip+venv first (the foundation!), then a uv-style graduation — then move into the professional world of tools! 🚀

Real-World and AI Engineering Relevance

Where this shows up in AI engineering

- import machinery and module resolution
- package structure and entry points
- virtual environments and dependency isolation

EXAM AUR INTERVIEW PERSPECTIVE

Agar koi concept interview ya exam mein poochha jaye, to definition ke saath apna ek chhota practical example zaroor bolo. Sirf theory batane se confidence nahi dikhta - implementation dikhane se dikhta hai.

Quick Reference

Short, representative snippets collected from the unit's notebooks. Use them as syntax reminders, not as complete programs.

PYTHON**1.2 MODULE PEEPING — `DIR()` AND DOCSTRINGS**

```
# 🔍 STEP-2: Module-peeping – dir() + docstring-self-guide! 📖
peek = [n for n in dir(gu) if not n.startswith("_")]
print("public-tools:", peek)
print("doc      :", gu.__doc__)

# tip: console me help(gu.average) likho – full-card milega! 💡
func_doc = gu.average.__doc__
print("average-doc :", func_doc)
# Expected OUTPUT:
# public-tools: ['average', 'grade_letter', 'topper']
# doc      :    grade_utils – Student-marks ke helper-functions ki Library! 📖
# average-doc :    Marks-list ka average nikaalo (round-1!).
```

PYTHON**1.3 THE IMPORT CACHE — `SYS.MODULES`**

```
# 📖 STEP-3: sys.modules cache – "already-load hai ji!" proof!
print("in-cache?   :", "grade_utils" in sys.modules)

# dobara-import: NAYA-execute NAHI – cache-se le-te! (speed-priya!)
import grade_utils as gu_2
print("same-object? :", gu is gu_2)
print("(cache-rule → import 2nd-time CHEAP; test-fixes require reload!)")
# Expected OUTPUT:
# in-cache?   : True
# same-object? : True
# (cache-rule → import 2nd-time CHEAP; test-fixes require reload!)
```

PYTHON**THE MEANING OF THE GUARD (THE GOLDEN RULE)**

```
if __name__ == "__main__":
    # these lines run ONLY when the file is executed directly
    # on the import side they are skipped – the module stays a silent library
```

PYTHON

REAL-WORLD HOOK: ORDER IN THE HOUSE-CUPBOARD

```

home/
├── kitchen/  ← a package! (___init___py = the index-card)
│   ├── daal_shelf.py  ← a submodule
│   └── masala_rack.py  ← a submodule
└── study/  ← a package!
    └── notes.py

```

BASH

TERMINAL-CASTLE — THE 3-STEP COMMAND MANTRA!

```

python3 -m venv myenv          # 1) kitchen-build (naye-folder!)
source myenv/bin/activate      # 2) ENTRY-pass 🚪 (prompt-start: (myenv))
python -V; which pip           # 3) INSIDE-proofs – venv-python vad-hi!
pip install numpy pandas      #   Libs isi-kitchen me!
pip freeze > requirements.txt  #   passport-export 📄
deactivate                    # 4) EXIT! (prompt normal!)

```

BASH

TERMINAL-CASTLE — THE 3-STEP COMMAND MANTRA!

```

python -m venv myenv
myenv\Scripts\activate      :: cmd-style
myenv\Scripts\Activate.ps1  :: PowerShell-style (run-policy care!)
python -V
pip install jupyter
deactivate

```

BASH

TERMINAL-CASTLE — THE 3-STEP COMMAND MANTRA!

```

python3 -m venv myenv && source myenv/bin/activate
pip install -r requirements.txt  ← purane-recipe saari-lib's back! 🔍

```

PYTHON

REAL-WORLD HOOK: THE EVOLUTION-CHART OF 3 TRANSPORTS

```

pip+venv = CYCLE 🚲  ← reliable, found everywhere, light!
conda    = SUV 🚗    ← heavy capacity; carries non-Python bags too!
uv       = BULLET-TRAIN ⚡  ← same city, but 10-100x faster (Rust magic!)

```

Revision Checklist

Work through this list without looking at the notes. Anything you cannot explain aloud belongs back in the revision queue.

☐ Real-World Hook: Hostel Mess Lunch 🏠

☐ 1. The Four Import Styles

☐ 1.2 Style 2 — `import module as alias`: Give It a Short Stamp

☐ 1.3 Style 3 — `from module import name`: Grab Only the Dish You Need

☐ 1.4 Style 4 — `from module import *`: The Wildcard Trap

☐ The Harm of `import *` (3-part warning!)

☐ 2. Inspecting Modules

☐ 3. The Shadow Trap — Never Name a File Like the Standard Library

☐ Summary

☐ Real-World Hook: One Recipe Book for Every Kitchen

☐ 1. Your First Module — Write, Point, Import

☐ 1.2 Module peeping — `dir()` and docstrings

☐ 1.3 The import cache — `sys.modules`

☐ 2. The Two Lives of a File

☐ 2.2 Running the same file as a script — `subprocess`

☐ The Meaning of the Guard (the golden rule)

☐ Real-World Hook: Order in the House-Cupboard

☐ 1. The Package Build — A Folder, an `__init__.py`, and Three Submodules

☐ 1.3 Three import styles for packages

☐ 2. Relative Imports — Wiring Inside the Package

☐ 3. The Dunder Zoo — How a Package Describes Itself

☐ Real-World Hook: The Kitchen-War of 2 Projects